Technical report — ParlaRadar, VectorZero

- **Challenge:** Topic classification for Swiss parliaments with Apertus (OpenParlData.ch, Digital Administration Switzerland use case)
- **Event:** On-site Liebefeld
- **Team:** VectorZero — Mitesh Kumar, Sagnik Das, Aswathy Marottikkal Ramesh, Prabin Sharma Poudel
- **Demo:** `https://youtu.be/v9TV0wexXJ8`

> Every figure below is copied from a result file committed in `evaluation/`. None is an independent accuracy benchmark; Section 5 lists what the numbers do **not** show.

1. Summary

Problem. OpenParlData.ch aggregates 300,000+ affairs from about 90 Swiss parliaments in German, French, Italian and Romansh. Topic tags differ between parliaments, and public-sector users such as Digital Administration Switzerland (DVS) track themes (for example "Digitalisation") that no official scheme contains. Journalists have a related problem: checking what a news story says about parliament against the record.

Approach. ParlaRadar is a local FastAPI application over a DuckDB copy of the OpenParlData archive. It uses Apertus for two tasks, always tied to source text:

1. **Classification** into the 165-topic Année Politique Suisse (APS) scheme or into a user-defined *custom* topic (for example the fixed DVS "Digitalisation" scope), without retraining. Every label must carry a quotation that is an exact passage of the source; otherwise the system abstains.
2. **News fact-checking.** A story is split into sentences; each is scoped (Swiss parliamentary or not), matched to archive passages, and its claims are marked *supported, contradicted, unverified* or *not factual*. Supported and contradicted verdicts need quotations selected from the archive and a second, independent model review. A missing match is never reported as "false".

Human review, abstention and verified quotes are design requirements. The default interface is a React app served by FastAPI at `/`; the original workstation stays at `/legacy`.

Measurement (headline results)	Result	Source
Recovery of 42 known DVS/PolitMonitor positives (Apertus 8B, prompt `v3`, 18 Oct 2026)	**20 / 42 = 47.6 %**, below a title-keyword baseline of 22 / 42 = 52.4 %	`evaluation/digitalisation-result.json`
Live news-check smoke set (Apertus 70B, 7 stories, 5 languages)	14 / 14 expected outcomes	`evaluation/news-live-regression.json`
Automated tests (mocked model)	74 passing	`python -m unittest discover -s tests`
Curia Vista proxy ("Medien und Kommunikation")	80.9 % recall, 40.9 % tag agreement	`evaluation/BENCHMARK.md`

There is **no precision, F1 or APS accuracy figure**: the digitalisation lists contain positives only and no affair-level APS ground truth exists.

2. Architecture

ParlaRadar is one FastAPI process (one Uvicorn worker) over one DuckDB file; Apertus is the only mandatory external service. The component diagram, data flows, retrieval design and storage model are in the **Architecture** section of the repository `README.md`.

Target architecture (mandatory)

None of the three target architectures has been deployed and tested by us. The demo and all measurements ran on one laptop (Apple M5, 17 GB RAM) against a hosted Apertus gateway and the public OpenParlData API. Every external dependency is a replaceable endpoint or a one-off download:

Target	Deployable?	How the constraint is met, and what is missing
a) On-premise	Yes, by configuration (untested)	One Python process, one DuckDB file, no database server. Set `LLM_BASE_URL` to an on-premise OpenAI-compatible server hosting Apertus (needs JSON-schema `response_format`). No Dockerfile is provided yet.
b) Air-gapped	Partially (untested)	Needs the archive, texts and embeddings built beforehand, the E5 model pre-cached (`HF_HOME`), Apertus served locally and live discovery off. Otherwise the app calls the OpenParlData API for missing texts, authors and events; those results degrade to *unverified*. The React UI loads Google Fonts and needs self-hosted fonts.
c) Sovereign Swiss cloud	Possible, not verified	As (a) on a Swiss cloud VM. The gateway we used has a `.ch` domain, but we did **not** verify its operator, jurisdiction or data residency. The optional Cloudflare Worker is non-Swiss and must not be used.

Dependency	Build time	Runtime	Replaceable by
OpenParlData exports / API	archive download	optional (missing texts, authors, events)	pre-imported archive
Multilingual E5 weights	yes	no (cached)	local model directory
Apertus endpoint	no	**mandatory**	any OpenAI-compatible Apertus server (`LLM_BASE_URL`)
Google Fonts (React UI)	no	yes (cosmetic)	self-hosted fonts

Privacy. Text a journalist pastes is sent to the Apertus endpoint and stored in the local DuckDB (`fact_checks`). On a hosted endpoint it leaves the machine, so use only non-confidential text.

3. Use of Apertus

- **Models:** `apertus-v1-5-8b` and `apertus-v1-5-70b` (selectable per run; default 70B; the judge always uses 70B). No fine-tuning, adapters or quantisation by us.
- **Use:** structured inference, multi-step chains (scope, assessment, independent review, correction) and evaluation (a blind LLM judge).
- **Where it runs:** a hosted endpoint, `https://gateway.keera.ch/api/v1`; we do not know its serving stack.
- **Configuration:** `LLM_NAME` (default model), `LLM_BASE_URL`, `LLM_API_KEY` (or a `.key` file). The earlier names `APERTUS_MODEL`, `APERTUS_BASE_URL` and `KEERA_API_KEY` remain fallbacks. The UI's 8B/70B selector sends the gateway names above.
- **Calls:** temperature 0, `max_tokens` 1800, JSON-schema `response_format` with `strict: true`. The schema enumerates the allowed topic and source IDs, so the model cannot return unknown IDs. One repair attempt on invalid output; one inference at a time (file lock).

Task	File	Version (part of every cache key)
APS / custom-topic classification	`classifier.py`	prompt `named-policy-evidence-v6`
Fixed DVS "Digitalisation" scope	`digitalisation.py`	`focus:digitalisation:v1`
News scope, assessment, review, correction, translation	`factcheck.py`	`archive-factcheck-v4`
Blind reference judge (70B)	`judge.py`	`blind-archive-judge-v1`

Design choices that follow from the model's limits. Embeddings shortlist 12 named topics out of 165 (when embeddings are imported). Quotes are verified by string matching in code: a quote the model invented cannot become a verified label. In the fact-checker the model returns evidence IDs and the

backend cuts the quote. Sources are marked as untrusted data; the model is told never to judge political positions; the classifier never receives the author or party.

4. Data

Data	Source	Use	Licence / status
Affairs, bodies, texts (326,397 affairs in our archive)	OpenParlData exports and API	Archive, retrieval	Public parliamentary records; the challenge material credits the Parliamentary Services via OpenParlData.ch but gives **no explicit licence text**: confirm before redistributing
Curia Vista areas, cantonal classifications	Challenge datasets	Source labels, never treated as APS ground truth	as above
APS scheme (8 domains, 22 chapters, 165 topics)	anneepolitique.swiss	Target taxonomy	Not stated; keep attribution
PolitMonitor "Digitalisierung" tags (3,433) and DVS list (30)	Challenge datasets	Benchmark positives	Editorial labels; positives only
Pasted news text	the user	Fact-checking	Stored in local DuckDB; sent to the Apertus endpoint

`data/`, `.key`, `.env`, model weights and large outputs are git-ignored; only small result files in `evaluation/` are committed.

Personal information. Affair records name the elected officials who authored an item, with party and role: the public record of their parliamentary work, shown as published. We collected no data from human subjects, so no consent procedure applied. Party is political information about identifiable people; we ran no party- or canton-level analysis.

5. Limitations

- **The only benchmark is unfavourable.** The 8B model recovered 20 of 42 known positives (47.6 %) against 22 for a title-keyword baseline (DVS subset 43.3 % vs 60.0 %; German 44.0 % vs 56.0 %; French 45.5 % vs 54.5 %; Italian 66.7 % vs 33.3 %). 22 of 42 ended in abstention, 10 had no substantive text, 20 had a rejected quote.
- **It pre-dates the current system.** It ran with prompt `v3`; the code is at `v6` and defaults to 70B. It was **not re-run**, so current recall on this set is unknown.
- **No precision, F1 or accuracy.** A seven-affair, AI-assisted sample (`evaluation/journalist-sample.json`) found 0.00 micro precision and recall for older 8B predictions: too small to generalise, but a warning against unattended use. A verified quote proves traceability, not that a label is right.
- **No fairness evaluation** (parties, cantons, languages) and no cross-lingual consistency test. Romansh quality is unverified.
- **The smoke test** (14 / 14, 7 curated stories) is not evidence of accuracy or of equal quality across languages.
- **The LLM judge is not ground truth:** a second Apertus 70B pass can share the model's mistakes, and cases it cannot assess are excluded.
- **Retrieval is bounded** (up to 8 passages, text fetched for 3 affairs); "unverified" means *not found*, never *false*. An allegation in the archive does not establish real-world truth.
- **Autonomous cost.** With an archive imported, start-up creates and runs a 300,000-affair classification (8B) and a 70B judge without a user action, and `GET /api/jobs` can restart it. `scripts/demo_server_safe.py` switches both off.

- **No authentication by default** (bind to localhost or set `PARL_BACKEND_TOKEN`). Fact-check history and review flags live in browser storage only.
- **Not tested:** on-premise, air-gapped and Swiss-cloud deployments; more than one API worker; the Slurm scripts on a cluster; vector-search latency on the full corpus.

6. Reproducibility

- **Code baseline:** `main` at `213fe88` plus this submission's changes (see `git log` for the final commit). Benchmark file produced 18 Oct 2026 with prompt `v3`.
- **Hardware:** Apple M5, 17 GB RAM, macOS (arm64); Python 3.13, Node 26; embeddings on Apple MPS (CPU fallback), optional Slurm script for one A40 GPU. Dependencies in `requirements*.txt` (partly unpinned); the embedding model commit is pinned in `data/embedding-revision.json`.
- **Determinism:** deterministic hashing for sampling, temperature 0 for model calls; answers still vary slightly with the gateway's model version.

```
python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements-
embeddings.txt
export LLM_API_KEY="<key>" LLM_BASE_URL="https://gateway.keera.ch/api/v1"
LLM_NAME="apertus-v1-5-70b"
python -m parliament bootstrap                      # import archive + APS catalogue (needs
network)
npm --prefix frontend ci && npm --prefix frontend run build
bash scripts/run_demo.sh                            # http://127.0.0.1:8000
python -m unittest discover -s tests                # 74 tests, mocked model, no credits
python -m parliament.digital_benchmark prepare && python -m parliament.digital_benchmark
run && python -m parliament.digital_benchmark report
```

Optional embeddings (vector search, 165-to-12 shortlist): `python -m parliament fetch --sample 200`, `export-units`, `embed`, `import-embeddings` (see the README). The benchmark and the live news check (`scripts/check_news_live.py`) spend Apertus credits and were **not** re-run for this report; their figures are copied from the committed result files.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. (The repository's source code is separately under the MIT licence, see `LICENSE`.)

References

- Challenge presentation: https://hackapertus-topic-classification-5c233f.gitlab.io/ · OpenParlData exports https://files.openparldata.ch/exports/ and API https://api.openparldata.ch/documentation
- Année Politique Suisse: https://anneepolitique.swiss/de/themen/ · DVS list: https://www.digitale-verwaltung-schweiz.ch/aktuelles/politische-geschaefte · DuckDB: https://duckdb.org
- In the repository: `evaluation/BENCHMARK.md`, `evaluation/digitalisation-result.json`, `evaluation/news-live-regression.json`, `evaluation/journalist-sample.json`, `examples.md`